



Prova de Recuperação (a₂)

17/10/2025

Nome: _____

Matrícula: _____

1 Instruções

1. Para fazer essa prova, você deve clonar o repositório no seu computador, resolver as questões abaixo e empurrar as soluções para o repositório remoto.
2. Ao clonar o repositório, é preciso instalar as dependências com o comando `npm install`.
3. Cada questão deve ser resolvida no arquivo correspondente, que está localizado no diretório `/questoes`.
4. Apenas os arquivos do diretório `/questoes` podem ser alterados. Não é permitido alterar outros arquivos ou diretórios.
5. O repositório já está devidamente configurado e não é necessário fazer nenhuma configuração adicional.
6. Você pode testar suas soluções localmente utilizando o comando `npm run test`. Esse comando executará todos os testes automatizados.
7. Não é necessário compilar o TypeScript manualmente, pois os testes já fazem isso automaticamente.
8. Caso queira executar os testes de uma questão específica, utilize o comando `npm run test:X`, onde X é o número da questão (1 ou 2).
9. A nota máxima da prova é 10 pontos.
10. É permitida a consulta somente ao material que está na pasta `/docs` e ao repositório dos exercícios de laboratório. Não é permitida a consulta a qualquer outro material, incluindo a internet.
11. A alteração de qualquer arquivo fora do diretório `/questoes` resultará em nota zero.
12. A detecção de plágio resultará em nota zero para todos os envolvidos.

2 Questões

1. (5 pontos) Utilizar **programação funcional** em **JavaScript** para implementar um sistema simples de gerenciamento de tarefas seguindo os requisitos mínimos (obrigatórios) abaixo. Outras funções podem ser implementadas caso necessário. Considere que cada tarefa tem o seguinte formato:

```
{  
  id: number,  
  titulo: string,  
  prioridade: "baixa" | "media" | "alta",  
  concluida: boolean  
}
```

O sistema deve ter as seguintes funcionalidades:

- a) Adicionar/atualizar tarefa:

- nome da função: `adicionarTarefa`
 - parâmetros: `lista` (lista de tarefas) e `tarefa` (objeto tarefa)
 - retorno: nova lista adicionando ou atualizando a tarefa
 - comportamento:
 - se a tarefa não estiver na lista, adicioná-la;
 - se a tarefa já existir, atualizar seus dados
- b) Remover tarefa:
- nome da função: `removerTarefa`
 - parâmetros: `lista` (lista de tarefas) e `id` (número do id da tarefa)
 - retorno: nova lista sem a tarefa removida
 - comportamento:
 - se a tarefa não estiver na lista, não alterar a lista;
 - se a tarefa estiver na lista, removê-la.
- c) Atualizar propriedade da tarefa:
- nome da função: `atualizarPropriedade`
 - parâmetros: `lista` (lista de tarefas), `id` (número do id da tarefa), `propriedade` (nome da propriedade a ser atualizada) e `valor` (novo valor da propriedade)
 - retorno: nova lista com a tarefa atualizada
 - comportamento:
 - se a tarefa não estiver na lista, não alterar a lista;
 - se a tarefa estiver na lista, atualizar a propriedade especificada com o novo valor, desde que o nome da propriedade esteja correto e o valor seja válido.
- d) Alterar status de conclusão:
- nome da função: `inverterConcluida`
 - parâmetros: `lista` (lista de tarefas), `id` (número do id da tarefa)
 - retorno: nova lista com a tarefa atualizada
 - comportamento:
 - se a tarefa não estiver na lista, não alterar a lista;
 - se a tarefa estiver na lista, inverter o valor da propriedade `concluida`.
- e) Buscar tarefa:
- nome da função: `buscarPorTitulo`
 - parâmetros: `lista` (lista de tarefas), `titulo` (string do título da tarefa)
 - retorno: objeto da tarefa encontrada ou `null`
 - comportamento:
 - buscar a tarefa que corresponda ao título fornecido
 - o título pode estar em maiúsculas ou minúsculas
 - se a tarefa não for encontrada, retornar `null`
- f) Listar resumos de tarefas:
- nome da função: `listarResumos`
 - parâmetros: `lista` (lista de tarefas)
 - retorno: lista de *strings* no formato "ID: [id], [titulo] ([prioridade]) - concluida: [sim/não]"
 - comportamento:
 - Caso a tarefa esteja concluída, o texto `concluida` deve ser `sim`; caso contrário, deve ser `não`.
- g) Limpar tarefas concluídas:
- nome da função: `limparConcluidas`
 - parâmetros: `lista` (lista de tarefas)
 - retorno: nova lista sem as tarefas concluídas

2. (5 pontos) Utilize **orientação a objetos** em **TypeScript** para modelar um sistema simples de gerenciamento de biblioteca seguindo os requisitos mínimos (obrigatórios) abaixo. Outros métodos e propriedades podem ser adicionados conforme necessário.

a) Uma classe abstrata `ItemBiblioteca`

- propriedades (protegidas):
 - `titulo` (string): título do item de biblioteca
 - `copias` (number): número de cópias disponíveis
 - `emprestimos` (uma lista de objetos do tipo `{ usuario: Usuario, dataEmprestimo: Date }`): registra os empréstimos atuais do item
- construtor: deve inicializar as propriedades da classe
- métodos:
 - `descricao`
 - * parâmetro: nenhum
 - * retorno: string
 - * comportamento: retorna uma descrição do item no formato "Título: [titulo], Cópias Disponíveis: [copias]"
 - `emprestar`
 - * parâmetro: `Usuario`
 - * retorno: boolean para indicar se a operação foi bem-sucedida ou não
 - * comportamento: permite emprestar se houver cópias disponíveis, decrementando o número de cópias em caso de sucesso, sendo que um usuário não pode emprestar o mesmo item mais de uma vez simultaneamente
 - `devolver` (abstrato)
 - * parâmetro: `Usuario`
 - * retorno: number

b) A classe `Livro` que estende `ItemBiblioteca`

- propriedade (privada):
 - `autores` (lista de objetos do tipo `Autor`)
- construtor: deve inicializar as propriedades herdadas e a propriedade da classe
- métodos:
 - implementar o método `devolver`
 - * retorno: valor da multa a ser paga (number)
 - * comportamento: incrementa o número de cópias e calcula a multa com base no número de dias atrasados (considerando uma taxa fixa de R\$ 2,00 por dia atrasado), sendo que se não houver atraso, a multa é zero; o usuário pode permanecer com o livro por até 30 dias sem multa;
 - * observação: caso o usuário não tenha emprestado o livro, o método deve retornar -1 para indicar erro.
 - sobrescrever o método `descricao`
 - * retorno: string
 - * comportamento: adicionar "Autor(es): [nome] [sobrenome] ([nacionalidade])" ao final da descrição herdada, separado por vírgula; todos os autores devem ser listados, separados por ponto e vírgula

c) A classe `Revista` que estende `ItemBiblioteca`

- propriedade (privada):
 - `edicao` (number)
- construtor: deve inicializar as propriedades herdadas e a propriedade da classe
- métodos:
 - implementar o método `devolver`
 - * retorno: valor da multa a ser paga (number)

- * comportamento: incrementa o número de cópias e calcula a multa com base no número de dias atrasados (considerando uma taxa fixa de R\$ 1,00 por dia atrasado), sendo que se não houver atraso, a multa é zero; o usuário pode permanecer com a revista por até 15 dias sem multa;
 - * observação: caso o usuário não tenha emprestado o livro, o método deve retornar -1 para indicar erro.
 - sobrescrever o método `descricao`
 - * retorno: `string`
 - * comportamento: adicionar "Edição: [edicao]" ao final da descrição herdada, separado por vírgula
- d) A classe `Usuario` que representa um usuário da biblioteca
- propriedades (privadas):
 - `nome (string)`: nome do usuário
 - `matricula (number)`: matrícula do usuário, que deve ser única
 - `matriculas (number[ ])` (estática): lista de matrículas já utilizadas para garantir unicidade
 - construtor: deve inicializar as propriedades da classe
 - métodos:
 - getters para `nome` e `matricula`;
 - getter estático para `matriculas`;
 - `gerarMatricula (estático)`
 - * parâmetro: nenhum
 - * retorno: número
 - * comportamento: gera uma matrícula única
- e) A interface `Autor` com as seguintes propriedades:
- `nome (string)`
 - `sobrenome (string)`
 - `nacionalidade (string)`